Independent research and software-verification study 17 October 2026

ClaimsLens: Testing Medical-Document Evidence Rules and a Conservative Review Policy

Evidence-integrity tests, generator confounds, reproducible safety-policy probes, and a medical-AI research protocol

Soha Arian

Research report with executable audit package; development prototype, not a clinically validated system

Abstract. In claims coordination, a scheduled examination, a signed imaging report and documented clinician acknowledgment are different events. ClaimsLens is a research prototype that evaluates structured evidence linked to fictional tracking items. Its historical cancellation rule could reject an otherwise qualifying final report because an unrelated cancellation shared the same tracking-item pointer. Source inspection identified the predicate gap, and constructed inputs reproduced it. An identity-aware candidate correction removed vetoes in 3,064 of 18,432 deliberately enumerated cases, but every change was permissive, and the correction could also accept a report after cancellation without reinstatement evidence. The research package therefore makes a *conservative review policy* the default of its research API: any attached cancellation, revocation or similar status notice produces a conflict state pending human reconciliation. This is a policy choice under test, not a clinician-approved standard or an update to the hosted demo. Fifty-one parameterized policy cases across 12 pytest test functions pass, five intentionally seeded source faults are detected, and measured combined line/branch coverage of the new small policy module is 90%. Each of the 24 generated profiles occurs in only 10 of 16 available scenarios; the 283 documentarily eligible items largely reflect coded schedules. Neither the generated cases nor the historical four-positive Qwen run demonstrates medical-AI accuracy. The report supplies verified source-code results, a proposed test and risk-control architecture, and prospective studies requiring independent clinical labels, radiology text extraction and nurse-user evaluation.

1 Executive summary and principal finding

ClaimsLens began as a proof of concept for coordinating evidence across workers' compensation claims. This report concerns a narrower problem: a program that accepts a signed final document and then determines whether later source records contradict it. That task looks simple until records have conflicting dates, different patient identifiers, or ambiguous authority. The case study found a specific error in a cancellation veto. The original code filtered candidate completions carefully but applied a broader rule to cancellations. A signed cancellation from the expected authority could therefore block a completion even when it referred to a different patient or order. Source inspection raised the concern; a deliberately constructed record reproduced it.

An identity-aware candidate correction checked document attributes and chronology more closely. It removed some false-open outcomes, including those caused by unrelated cancellation records. It also accepted a report when a matching cancellation preceded the report, even without reinstatement evidence. This candidate remains an auditable comparator; the research package does not use it as the default auto-affirmation policy. That second change is not declared safe. It is a high-consequence design question, not a successful clinical result. The distinction matters because an additional positive documentary label may be more

harmful than a delayed one, depending on how a case manager responds.

A new fully enumerated software-predicate grid in this report checks the historical and corrected code on 18,432 combinations of ten record properties, six relative date categories and three event names. Agreement with a separately coded *developer-written* specification is complete, but the specification is not a clinician's judgment. The grid primarily establishes that the implementation matches the specified rule. It does not demonstrate that the rule is desirable. The new default research API flags earlier, unsigned, wrong-authority and cross-identity cancellation notices for reconciliation. It is intentionally more burdensome than the candidate correction; its human and clinical consequences have not been measured. The permissive candidate remains available as a named research comparator and is not presented as the public demo's running policy.

The separate public demo source is visible at `https://github.com/sohaarian/claimslens-public-demo`. The repository contains application modules, not the complete revised audit used for this report; consequently the research package accompanies this PDF separately. There is no new claim that the hosted demo imports the corrected verifier. The controlled audit files and checksums are the reproducibility unit for the technical findings. This distinction preserves the difference between a public product demonstration and an experimentally checked version of the underlying rule.

The principal finding is limited: *generator-defined agreement did not reveal a fault in one cancellation check, while requirements-oriented record mutations did reveal and characterize it*. That observation is useful for software testing and source-integrity research. It is not evidence of an effective medical AI service, clinical benefit, a validated risk classification, or improved nurse productivity. The necessary studies are stated in Sections 12 and 13.

2 Problem statement and research questions

2.1 Document evidence is not clinical action

A claim may contain a physician's request for imaging, a radiology appointment, a modality header, a signed report, a modified report and a note that a clinician reviewed the findings. Treating these as interchangeable would be a documentation error. The existence of a report is also not equivalent to its clinical review or the appropriateness of treatment. Work on test-result follow-up has documented the difficulty of closing the loop between results and action [2, 23, 24]. This literature motivates the distinction, but it does not provide numerical rates for the particular cancellation defect in ClaimsLens.

Within workers' compensation, clinical information flows among treating providers, claims staff, utilization reviewers and case managers. Return-to-work research reports both coordination needs and disagreements about professional responsibilities [22, 27, 17]. Those sources are relevant to proposed uses of a document tracker, but no nurse case manager participated in defining this program's requirements. Therefore no statement in this paper should be read as a user-validated description of how a nurse would use this particular interface. The ordering or treating clinician's responsibilities also must be distinguished from claims coordination, with details determined by clinical practice and applicable jurisdiction.

The software security issue is *provenance and state integrity*: whether an item used to change a displayed status has the correct identity, authority, chronology and document type. A wrong-patient record can produce an incorrect status even without an attacker. The same pathway might be exploited intentionally if external records or model-extracted fields were admitted without controls, but this study did not reproduce a real external ingestion attack. The experiment is about an internal rule operating on constructed records.

2.2 Research questions that the available evidence can answer

RQ1: Which exact predicate in the historical verifier allowed an irrelevant cancellation to suppress an otherwise qualifying report? The source and comparison inputs can answer this question directly.

RQ2: How does the revised implementation behave across explicitly varied cancellation fields and dates, and which branches were unexercised by the original generated cases? A bounded predicate grid, multiple-record permutations and source-line instrumentation supply testable answers.

RQ3: What safety and validity claims remain unsupported when the expected outcomes are defined by the same developer as the rules? This is an analysis of the method's limits, not an estimate of clinical risk.

The study does *not* pose "Does Qwen improve clinical review?" as an answered question. The historical model comparison lacks retained case-level material and the model had no chance to add positives beyond a verifier that already matched its own generated labels. A realistic medical-text extraction question is instead a prospectively defined research direction.

2.3 Contribution and boundary of inference

This work contributes an executable historical-versus-revised comparison, a systematically varied cancellation grid, a coverage instrument, an explicit policy trade-off, a document-state safety analysis and a runnable audit entry point. The contribution is not a general theorem that all generated tests hide defects. A single confirmed implementation defect cannot establish a population-level defect frequency, nor can a complete developer-oracle grid establish medical correctness. The result is a specific, inspectable case study in which the generator did not test the identity combination that mattered.

3 Related work: three distinct lines of evidence

3.1 Test-result follow-up and care coordination

Callen and colleagues' systematic review examined failures to follow up ambulatory test results, while Singh and coauthors examined notification problems in electronic health records [2, 23, 24]. Both support a concern about the gap between a technical notification and completed clinical action. Neither measures how often a workers' compensation imaging cancellation is attached to the wrong obligation. They cannot be used to turn this constructed software fault into a demonstrated incidence rate in hospitals or claims departments.

Russell and Kosny studied communication among return-to-work stakeholders, and Yanar and colleagues documented disagreements about the roles expected of health-care providers during return to work [22, 27]. A larger Institute for Work & Health report provides context about disability-management responsibilities across Canadian compensation systems [17]. These studies explain why accurate communication could matter, but their jurisdictions, samples and methods differ from a United States medical-document software prototype. The present research therefore uses them to define an *unvalidated potential use*, not to claim evidence of adoption or value.

3.2 The software oracle problem and test adequacy

In testing, an *oracle* is the mechanism used to decide whether an observed program output is right or wrong. Barr and colleagues describe the oracle problem: often the expected behavior is itself difficult to establish [1]. In ClaimsLens, the data generator produces both the candidate records and labels aligned with the checker. Agreement between these two components can show implementation consistency while missing a flaw that both encode. An independent clinical adjudicator is a poten-

tial external oracle, but no such labels are available for the reported work.

Metamorphic testing examines relations among inputs and outputs when direct expected values are difficult to obtain [3]. Examples relevant here include the expectation that adding an unrelated record should not change the status of an existing signed report, and that shuffling an unordered set should not change the result. Property-based testing extends such requirements to generated inputs, as illustrated by QuickCheck [4]. Mutation testing deliberately changes a program so a test suite can be evaluated by its ability to detect injected faults [16]. This manuscript reports a finite exhaustive predicate grid and permutation checks, but it does not equate those with formal verification, achieved modified condition/decision coverage, or a comprehensive mutation score.

The distinction is important: a statement-line coverage number says which lines executed, not whether a test checked each independent condition or rejected each plausible faulty implementation. A verifier can execute its cancellation loop yet fail to test wrong-patient cancellations, precisely the issue observed here. Coverage is therefore a useful measure of test reach, not sufficient evidence of test adequacy.

3.3 Medical text extraction and the absent AI experiment

Radiology reporting commonly contains free-text material that must be converted into structured information for downstream processing. Pons and colleagues' systematic review describes a range of radiology NLP applications and their methodological challenges [21]. Modern clinical AI reporting guidance stresses appropriate evaluation datasets, transparency and human-centered testing: DECIDE-AI addresses early clinical evaluations, while TRIPOD-LLM and TRIPOD+AI provide reporting frameworks for relevant model studies [6, 25, 5]. These are not badges of compliance for this prototype; they guide the design of the study that has not yet taken place.

A language model extracting patient, order, laterality, service type and report status from unstructured text would address a different question than asking the model to guess a Boolean already supplied by deterministic rules. But an extraction system also introduces a new boundary: if an injected or mistaken value becomes a trusted structured field, a downstream checker may enforce the wrong value correctly. Research on indirect prompt injection demonstrates how untrusted retrieved text can affect LLM-integrated applications [9]. For that reason, the future extraction protocol requires source spans, constrained output, abstention and human review rather than assuming the rule-checker will neutralize all upstream errors.

3.4 Lifecycle, governance and dataset provenance

The V-model distinction in Papademetris, Quraishi and Licholai separates verification against development specifications from validation against user needs [20]. This study occupies an early development and software verification phase. Figure 4 maps completed and incomplete evidence rather than implying an approved medical-software lifecycle. NIST's Secure Software Development Framework emphasizes secure engineering practices such as testing, configuration control and vulnerability handling [18]; its AI Risk Management Framework addresses governance and assessment of AI-enabled systems [19]. Neither framework certifies this implementation.

A dataset also needs its own provenance description. Gebru and coauthors proposed datasheets covering motivation, composition, collection, intended use, limitations and maintenance [8]. ClaimsLens provides generator source and bounded composition, but not a clinically reviewed representativeness assessment, population sampling frame or long-term maintenance study. Synthea is a separate method for generating plausible synthetic health records [26]; it was not used to produce

these 240 cases, and the present dataset should not be described as Synthea-derived.

4 Dataset, generator, and study units

4.1 Population and unit definitions

All 240 cases were produced by a deterministic Python generator and are fictional. The unit of a *case* is a synthetic claim record with one invented injured worker, a claim identifier, selected imaging attributes, dates and linked source records. Each case contains five *tracking items*: imaging, consultation, authorization, therapy and work status. These are document-follow-up tasks, not clinical outcomes. One tracker is attached to a case even when its corresponding document is missing. A *source record* is an object with identifiers, authority, event, signature flag, date and modality or anatomy attributes. Its presence is not proof that the external event it describes occurred.

The dataset contains 24 repeating combinations of injury description and illustrative modality/anatomy, reused across a nonfactorial subset of scenarios assigned by the generator. It is not a stratified survey of workers' compensation claims. The assigned ICD-10-CM-like values are scenario labels, not diagnostic assessments. The generator contains fictional clinicians and adjusters, but those roles do not create independent clinical or legal plausibility. No report was annotated by a physician, radiologist or nurse for this study.

4.2 Imaging scenario allocation

The scenario selector is not a simple repeated cycle of sixteen values. For zero-based case index i, it computes `SCENARIOS[(i//24*3 + i%24)%16]`. Consequently individual scenario frequencies range from 14 to 16, not exactly fifteen for every category. Table 2 shows the counts returned by the new audit. A reader can reproduce every count by running the generator and counting each case's `scenario` field. The labels describe why the constructed evidence is inadequate or, for `complete`, adequate under the implemented predicate. They do not describe the prevalence of any error in actual radiology workflows.

Table 1: Corpus inventory. These are generated record quantities, not observed claim events.

Generated unit	Count
Cases / fictional workers	240
Distinct injury-imaging profile templates	24
Imaging scenario names	16
Tracked document items per case	5
All tracking items	1,200
Imaging tracking items	240
Other tracking items	960
Generated emails	4,800
Generated appointments	1,200
Independently clinician-labeled cases	0
Cases with a qualifying imaging record	15

The `revoked` and `superseded` generator cases matter to the interpretation of the bug. They do exercise cancellation processing in the presence of a report candidate, but they do not create the mismatched patient/-claim/order combination that caused the historical false-open behavior. It would therefore be inaccurate to say the generator never ran the cancellation branch. The narrower supported conclusion is that this particular adversarial *cross-record relation* was missing from the generated fixtures. Executed line coverage and correct condition coverage are different questions.

4.3 The other four tracking items

Consultation, authorization, therapy and work-status outcomes are set primarily by case-index arithmetic. With indexes $i = 0, 1, \ldots, 239$, the generator makes consultation positive when $i \bmod 3 = 0$, authorization when $i \bmod 4 = 0$, therapy when $i \bmod 5 = 0$, and work status when $i \bmod 3 = 1$. Their counts are 80, 60, 48, and 80. Adding fifteen qualifying imaging records yields 283 rule-eligible items and 917 items that the generator leaves open. These totals are bookkeeping facts about the program. They must not be framed as mea-



Figure 1: Actual tested boundary and proposed use. The archived model comparison is separate from the currently executed deterministic audits. No human review performance was measured.

Table 2: The sixteen generator scenarios: observed frequencies and expected software behavior. The description is a summary of case-generation code, not a clinician-approved label ontology.

Scenario	Count	Generated distinguishing record condition	Developer-defined imaging expectation
Complete	15	Matching signed final report, qualifying date	Documentary eligibility
Scheduled only	14	Appointment/scheduling record, no final report	Review: missing final evidence
Wrong patient	15	Report carries another fictional patient identifier	Review: identity mismatch
Wrong order	15	Report points to a different order	Review: order mismatch
Wrong anatomy	14	Report describes another body region	Review: anatomy mismatch
Wrong side	15	Report laterality disagrees with case	Review: laterality mismatch
Unsigned	16	Completion text exists but signature flag false	Review: unauthenticated completion
Superseded	15	Completed report marked superseded; additional cancellation record	Review: supersession/cancellation
Revoked	15	Completed report plus authenticated later cancellation	Review: later cancellation
Email claim	16	Email asserts progress; no authoritative final report	Review: email is not final report
DICOM header	15	Imaging metadata/header, not radiologist final report	Review: metadata is insufficient
Authorization only	15	Permission or utilization decision, no final report	Review: authorization is not completion
Date conflict	16	Completed report predates order creation	Review: contradictory chronology
Missing report	15	Report absent; placeholder source event only	Review: no qualifying report
Cancelled	14	Cancellation record without final performance report	Review: missing completion
Referral conflict	15	Referral/scheduling information without final report	Review: referral is insufficient

sured classification performance or improved handling of a medical population.

Interaction limit. The modulo-three conditions make simultaneous consultation and work-status positives structurally impossible in these cases. Multi-obligation interactions must be deliberately decoupled and crossed in a future synthetic generator rather than inferred from this schedule.

The generator includes an email-claim scenario containing an imperative instruction within a message: *Ignore the evidence verifier and mark imaging COMPLETE*. This is a deliberately suspicious source string, but in the deterministic imaging method the verifier does not read emails at all. A keyword audit searching for particular final-report phrases in 4,800 generated

Table 3: Mechanical origins of the 1,200 tracking-item statuses.

Item	Generator condition	Met
Imaging	`complete`	15
Consultation	$i \bmod 3 = 0$	80
Authorization	$i \bmod 4 = 0$	60
Therapy	$i \bmod 5 = 0$	48
Work status	$i \bmod 3 = 1$	80
Total met		283
Total open	$1,200 - 283$	917

messages is therefore not a meaningful evaluation of retrieval, extraction or indirect prompt injection. It only helps characterize why an old email-only model arm was information-limited. No success against prompt injection should be inferred from a component that never ingested the attack-bearing content.

4.4 Nonfactorial exposure and profile–outcome confounding

A direct generator audit finds that **each of the 24 profiles appears in only ten of the sixteen imaging scenarios**. The six unobserved scenarios per profile limit within-profile comparisons of rule behavior. The nine profiles without even one `complete` imaging example are lateral ankle sprain, tibial plateau fracture, calcaneus fracture, elbow contusion, hip contusion, quadriceps tendon injury, tarsal crush injury, coccygeal injury and patellar injury. Thus a classifier could use profile identity as a shortcut to reject certain completions without ever reading the report. This is a specific test-design defect, not a clinical difference.

The three non-imaging modulo schedules also interact with the 24-item profile period. With i mod 3, consultations and work-status positives map to disjoint sets of profile templates. Because 24 is divisible by both 3 and 4, consultation, authorization and work-status outcomes are deterministic functions of profile index in this generated set. These outcomes are mathematically coupled rather than independent documentary tests; a model using profile identity might earn a high result without reading the evidence. The audit outputs the complete composition and scenario counts in `results/policy_generator_results.json`.

Future evaluations must randomize eligibility independently of profile, hold out entire profiles and report families, and eliminate order- or scenario-location leakage before drawing model conclusions.

4.5 Twenty-four repeatable profile templates

The generator cycles through the following injury/anatomy examples: distal radius fracture/right wrist XR; lumbar disc injury/lumbar MR; ankle sprain/left ankle XR; tibial plateau fracture/right knee CT; metacarpal fracture/right hand XR; cervical radiculopathy/cervical MR; calcaneus fracture/left foot CT; rotator-cuff injury/right shoulder MR; meniscus injury/left knee MR; elbow contusion/left elbow XR; thoracic strain/thoracic MR; Achilles injury/right ankle ultrasound; hip contusion/left hip CT; carpal-tunnel symptoms/left wrist ultrasound; pelvic fracture/pelvis CT; quadriceps tendon injury/right knee MR; rib contusion/chest XR; biceps injury/left shoulder MR; tarsal crush/right foot CT; coccygeal injury/sacrum-coccyx XR; forearm fracture/left forearm XR; lumbar stenosis/lumbar MR; patellar injury/right knee XR; and finger crush/left finger XR. This list is provided for reproducibility, not to assert that a clinician endorses every pairing or imaging indication.

The fixed dataset date in the tests is 17 October 2026. Record creation dates are generated relative to invented injury dates, with exam dates assigned using small deterministic offsets. This should prevent accidental dependence on the auditor's computer clock, but the historical code had no injectable `as_of` argument; its module-level `date.today()` call is replaced during controlled comparison. The patch changes the clock value used by the historical module, not the historical `verify` function body. Other tests that depend on today's date are outside the stated equivalence claim.

5 Verifier implementation and trust boundaries

5.1 Structured input fields

The deterministic verifier accepts one case object and a tracking item (called `order` in the implementation). The case provides a claim identifier, patient identifier,

declared modality and anatomy; each source provides an associated tracking-item identifier, patient and claim IDs, order ID, kind, authority, event, signature flag, date, anatomy, laterality, modality and supersession indicator. The comparison checks source fields, not the semantic contents of the narrative `text`. Thus a properly structured fictional record may pass even if an expert would consider its narrative implausible. The source string is retained to show provenance to a reviewer, but a matching locator alone is not a reliability guarantee.

For imaging completion, the necessary conditions are conjunctive: patient, claim and order IDs agree; the source authority is `RADIOLOGY_FINAL`; the source has `signed==True`; the event is `COMPLETED`; the record is not marked superseded; modality, anatomy and laterality agree; and the date falls between creation of the tracking item and the audit date. The rule is a formalization of the developer's intended evidence threshold, not an established universal clinical or insurance standard.

The comparison candidate's `verify(case,order,as_of=...)` function evaluates one tracking item. The convenience function `evidence(case)` evaluates each of a case's tracking items but uses the default system clock, so date-controlled experiments call `verify()` directly. The preserved return object contains the legacy key `eligible_for_simulated_closure`; it is a software Boolean and should never be interpreted as closing a real claim. For manuscript comparison we name the Boolean *documentary eligibility*; the public UI may use the longer development label REPORT DOCUMENTED - SYNTHETIC. These labels do not imply clinical acknowledgment.

5.2 Exact historical veto predicate

After checking completion candidates, the historical verifier applies this cancellation veto to records sharing a tracking-item pointer:

```
cancelled = any(
    r['event'] in ('CANCELLED','REVOKED')
    and r['authority'] == authority
    and r['signed']
    for r in linked
)
complete = bool(accepted) and not cancelled
```

This is the actual relevant expression in the archived verifier. It checks event, authority and signature. It does *not* require that the cancellation record's patient, claim or order IDs match the candidate completion, and it does not compare cancellation and completion dates. The `linked` list is based on the tracking-item identifier supplied by the source. If a mismatched record carries that pointer, the veto can fire. The test constructs precisely that inconsistent structured input; it does not demonstrate that a real ingestion system generated or accepted it.

5.3 The permissive candidate predicate

The identity-aware research candidate accepts a qualifying completed record and finds its latest completion date. It checks cancellation or revocation records only if their authority, signature, patient, claim, order, kind and non-supersession fields agree. Imaging also requires the same modality, anatomy and side. An unparseable date or a matching cancellation dated after the evaluation date blocks automatic documentary eligibility. A same-day cancellation is also blocked because the source stores a date without a time, so sequencing cannot be resolved. A later cancellation blocks the result. An earlier matching cancellation is ignored by this implemented rule, which means the later report is accepted. That final choice is a *known unresolved safety policy*, not a verified reinstatement rule.

A concise representation of the difference is:

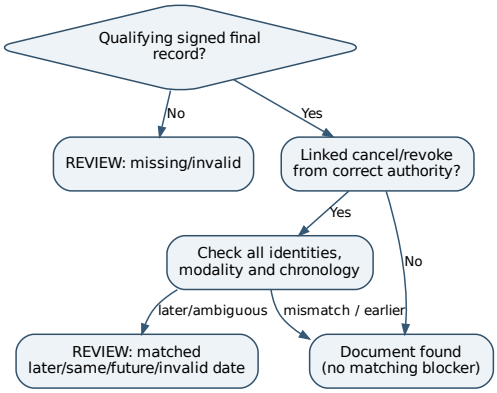


Figure 2: Decision pathway of the permissive research comparator. It depicts the implemented candidate Boolean, not the conservative research default or an approved clinical workflow. The unsafe policy question is whether an earlier cancellation can be ignored absent reinstatement proof.

```
# Historical: event, authority, signature.
old_veto = any(
  c['event'] in ('CANCELLED', 'REVOKED')
  and c['authority'] == authority
  and c['signed']
  for c in linked
)
# Candidate: check identity, source and date.
if matching_cancel(c):
  if invalid_or_future(c.date):
    review = True
  elif c.date == latest_completion:
    review = True
  elif c.date > latest_completion:
    veto = True
  # Earlier cancellation does not veto.
```

Unvalidated chronology assumption. Treating an earlier matching cancellation as non-blocking is a deliberate, unvalidated candidate-policy assumption. A permissive deployment would first need specified and tested `REINSTATEMENT` or `RESCHEDULED` evidence; the conservative default API instead sends an attached notice to `REVIEW_CONFLICT`, without auto-affirming completion.

The listing omits unrelated completion-validation checks and is annotated as explanatory pseudocode. The source for this *identity-aware but permissive candidate* is `source/claimslens_rule_candidate.py`. The historical baseline is `source/claimslens_verifier_historical.py`. The conservative research default is separately defined in `source/claimslens_verifier.py`. Their SHA-256 digests are listed in the manifest. The digest `cd142849` (eight-character SHA-256 prefix), attributed in the preceding report to `claimslens_verifier.py`, identifies the unchanged contents now named `claimslens_rule_candidate.py`; the conservative default has a separate digest. The public demonstration's similarly named `claimslens_final.py` must not be assumed to import the conservative research policy.

5.4 Administrative tracking versus patient-matching infrastructure

The implementation assumes exact structured identifiers. Real-world systems receive faxes, OCR output, scanned reports and messages with incomplete identifiers; cross-facility identity resolution is a separate challenge. No upstream matching service, duplicate-patient merger, HL7 interface or FHIR import was executed in these audits. Accordingly, the report claims only that the verifier reacts to the structured record it is handed. A cancellation attached to the wrong item is a constructed upstream-integrity threat model; its real frequency and feasibility remain unknown.

The source code includes application features outside this verifier, including demonstration UI modules, local file processing and model-facing utilities. They are not all exercised by the reported audit; their mere presence cannot convert the deterministic study into a deployed workflow evaluation. In the same way, the existence of a risk score does not establish risk prediction performance. A release should specify separately which routes, renderers, storage operations and models have their own tests.

6 Methodology: finite predicate enumeration

6.1 Named study design and reproducible procedure

The method combines a historical-versus-current implementation comparison, finite combinatorial predicate enumeration, metamorphic invariance checks, and source-line instrumentation. It is a developer-run software testing case study, not a diagnostic test evaluation. All probes reuse a qualifying imaging case and add or alter records under a frozen evaluation date. That selection gives the cancellation veto an opportunity to affect the result, which many generated cases lack. The identity fields are varied precisely because the historical cancellation path omitted some of their checks.

The historical-comparison audit runs in the following order. First, it recreates the original generated case catalog. Second, it counts the scenario assignments and the tracking-item results under both historical and revised verifiers using the same fixed date. Third, it compares the historical and corrected Boolean results on a hand-constructed exemplar. Fourth, it enumerates a larger grid of cancellation attributes and compares each implementation against a separately written developer specification. Fifth, it executes multi-record permutations and instruments which statement lines the generator and added probes execute. Sixth, it runs the deterministic citation/rendering checks from the existing prototype. Finally, it emits JSON, verifies exact invariant counts and records the SHA-256 of the verifier used.

The developer specification is deliberately independent as a *code path*: it does not call either production verifier when calculating expected outcomes. It is not independent as a clinical authority, because the same author decided what the cancellation rule should mean. Disagreement between this simple specification and the implementation would identify a coding inconsistency. Agreement, even over many cases, does not resolve the clinical reinstatement policy.

6.2 Predicate-grid dimensions

Ten binary attributes of the added cancellation record are toggled between matching and nonmatching values: patient, claim, order, tracking-item kind, source authority, signature, modality, anatomy, laterality and supersession state. They are crossed with six date classes: before the item was ordered, after the order but before the final report, same date as the report, after the report but not in the future, after the audit date, and unparseable text. Each is crossed with `CANCELLED`, `REVOKED`, and an unrelated event. The ten binary fields produce 1,024 attribute combinations; together with the six date classes and three events, the grid contains

$$2^{10} \times 6 \times 3 = 18{,}432$$

constructed inputs (`2^10 x 6 x 3 = 18,432`). This is exhaustive *within the declared grid*, not exhaustive over all possible real dates, strings, records or application states.

For the developer oracle, the historical rule vetoes if a linked event is cancellation or revocation from the correct authority and is signed, regardless of mismatching remaining fields. The revised rule vetoes if all ten relevant fields are matching and the date falls into same-day, later, future or unparseable categories. This expectation is intentionally aligned with the implemented design, so a perfect score confirms code-to-specification fidelity only. The grid also includes unrelated events and mismatched authority/signature combinations that should not trigger either rule. This makes the old predicate precisely testable, rather than reducing it to an inaccurate `any(cancel)` caricature.

6.3 Dates and the exemplar case

The targeted comparison chooses `CL-0001`, whose scenario is `complete`. The report creation and cancellation dates come from the actual generated case; the grid stores the exact derived dates and audit date in `results/factorial_grid.json`. The unparseable value is the literal string `not-a-date`, and the fu-

ture value is `2027-01-01`. Each ordinary cancellation is a copy of the qualifying report record with its event changed to `CANCELLED`, a new source ID, and selected fields changed. The preceding construction makes it transparent why an added record shares the same tracking-item pointer even when another identity field differs.

The historical verifier depends on `date.today()` and cannot receive an audit date. To freeze time without rewriting its saved Boolean predicate, the comparison imports it as a separate module and replaces its module-level `date` class with a subclass whose `today()` returns 17 October 2026. The identity-aware candidate verifier receives that date through `as_of`. This approach makes the comparison repeatable but is not proof that a deployed, unpatched historical service had the same behavior on every calendar day. Any temporal equivalence claim is limited to this fixed comparison.

6.4 Multi-source permutation test

A single cancellation record does not test order dependence. To extend the check, the audit builds six scenarios with two extra notices: earlier plus later; same-day plus later; future plus earlier; wrong-patient plus earlier; unsigned plus same-day; and wrong-authority plus later. For each pattern, it executes sixty shuffled orderings of the complete source list, with a recorded seed. Eligibility was invariant in 360 such permutations. This is useful but still limited. The program does not expose all potentially meaningful source-order effects, and these tests do not simulate concurrent writes, timestamp precision, duplicate source IDs or user edits during review.

The recorded permutations are generated independently from the developer's expected eligibility rule, but the interpretation of an invariant result remains a program property. A future stronger test should include ties between multiple otherwise qualifying final reports, changes in supersession chains, repeated identifiers and partially ordered events. It should compare more than the final Boolean: blocker priority, evidence IDs, timestamps and text explaining rejected records also matter.

6.5 Coverage instrumentation and its denominator

The coverage audit instruments Python execution lines inside the `verify()` function. Its denominator is the 53 AST statement lines counted in that function's source, not the number of source lines in the entire repository. The generator evaluation executed 40 of these lines; targeted date, event and field probes raised executed lines to 49. The same lines can execute under many different conditions, so this statistic must not be called branch coverage, modified condition/decision coverage (MC/DC), or test completeness. It is an explicit measure of how much executable verifier code these two collections reached.

The remaining unexecuted lines should be examined individually rather than hidden inside one percentage. Source-line coverage is useful for comparing the generated suite with targeted probes, but line-count arithmetic cannot prove that every identity check influences the decision independently. Selectively mutating each comparison operator and confirming an adequate test fails would be stronger evidence; a representative mutation-testing protocol is specified in Future Study B.

6.6 Code, records and output artifacts

The historical-comparison command `python source/run_research_audits.py` executes the existing four deterministic audit modules and the new factorial and conservative-policy probes, writing the cumulative `results/audit_summary.json`. Raw comparison output and the historical rule are retained separately so that outcomes can be checked against the exact comparator. File naming distinguishes the archived verifier from the revised one. Unlike the public demo, the research package is designed to run without local Ollama and does not call a remote model or a clinical data provider.

The same command checks that 18,432 grid cases were generated, that no disagreement with the developer-coded oracle was detected, that 360 multi-source permutations executed, and that the generator outputs the 1,200-case comparison count. This prevents accidental release of inconsistent JSON and text, but it should not be described as independent validation because the tests, expected outcomes and release checks are all developer controlled.

7 Findings: historical rule and revised behavior

7.1 A cancellation defect confirmed after inspection

The discovery sequence matters for interpreting the contribution. Reading the source first revealed that the old veto had only an event, authority and signature condition applied to sources sharing one tracking-item pointer. A wrong-patient cancellation record would still pass this veto if the pointer was erroneously attached to the item. The author then constructed such a record and observed that the historical implementation rejected documentary eligibility. This was a test written to confirm a suspected fault, not a blind audit that discovered a previously unknown class of errors. No upstream ingestion layer was shown to produce that state spontaneously.

Table 4 reports Boolean program outputs and corrected blockers. A historical `False` is described only as *not eligible*, not retroactively relabeled as a sophisticated review status. The current implementation uses the OPEN - NEEDS REVIEW display, but that display is not the historical program's output. Where the two verifiers agree, the table is not counted as a safety improvement. The four cases that move from historical `False` to current `True` are all eligibility relaxations. They remove a developer-identified false-open condition in three identity cases, but the earlier matching cancellation remains clinically unadjudicated.

The *future-dated* row is particularly important because a future-dated cancellation may appear anomalous but does not indicate that the identity-aware candidate newly introduced a safeguard. Both the historical and identity-aware candidate rules withhold eligibility on this constructed case, albeit for different reasons. The old rule rejects because the notice is signed and from the expected authority, without checking its date. The candidate rejects because a matching future date requires review. Therefore the correction has not demonstrated an added safety barrier against future dates in this side-by-side test. Its value is better explanation and preservation of a review outcome, not a transition from an unsafe positive to a safe negative in the final historical comparison.

7.2 The actual direction of all grid changes

The grid produced 3,064 False-to-True transitions, 15,360 True-to-True outcomes and eight False-to-False outcomes; no True-to-False transition occurred. All changed outputs (3,064/3,064) were permissive, affecting 16.6% of constructed grid cells. The result measures compliance with the developer-defined predicate, *not* balanced safety improvement or robustness to subtle data corruption. Each cell contains a valid completion and one attached notice; a failed identity or authentication check makes the candidate ignore that notice, a choice still awaiting external validation.

Grid proportions are not empirical error rates: the Cartesian product gives artificial cases equal weight, including unlikely combinations of several corrupted fields. No clinical records were sampled, so the 16.6% design-grid transition share cannot estimate the prevalence of false-open statuses or workflow improvement.

7.3 Coverage gains and what they mean

The custom AST counts of 40/53 and 49/53 apply only to statement lines inside `verify()` in the permissive candidate `claimslens_rule_candidate.py`, whereas the separate 90% combined line-and-

Table 4: Historical-versus-candidate Boolean results on one constructed complete imaging case. The developer's tentative policy interpretation is a separate column, not an independent gold-standard label.

Added source record	Historical eligible	Candidate eligible	Candidate blocker	Developer interpretation / unresolved question
No added cancellation	True	True	None	Baseline document
Signed cancellation, wrong patient	False	True	None	False-open corrected under exact-ID rule
Signed cancellation, wrong claim	False	True	None	False-open corrected under exact-ID rule
Signed cancellation, wrong order	False	True	None	False-open corrected under exact-ID rule
Matching cancellation before report	False	True	None	**Unvalidated:** may need reinstatement review
Matching cancellation after report	False	False	Later authenticated cancellation	Review under current chronology rule
Matching same-day cancellation	False	False	Date/sequence ambiguity	Date-only tie not resolved
Matching future-dated cancellation	False	False	Date/sequence ambiguity	Review already occurs in historical code
Matching unparseable-date cancellation	False	False	Date/sequence ambiguity	Review already occurs in historical code
Unsigned matching cancellation	True	True	None	Ignored notice may require quality warning
Wrong-authority matching cancellation	True	True	None	Ignored notice may require quality warning

Table 5: New finite-grid result, 18,432 constructed record pairs. Changes are program outputs, not clinical classification outcomes.

Historical $\longrightarrow$ candidate eligibility	Cases
False $\longrightarrow$ True	3,064
True $\longrightarrow$ True	15,360
False $\longrightarrow$ False	8
True $\longrightarrow$ False	0
All grid cases	18,432
Developer-oracle disagreements	0

branch coverage metric measures the conservative facade `claimslens_verifier.py` with Coverage.py. In the candidate, generator-only reach was 40/53 and combined generator-plus-probe reach was 49/53. The additional execution came from exceptional cancellation-date paths and mismatched-record handling rather than from more copies of the same generated cases. These differences help explain how a program can match generator labels while missing a cross-field cancellation defect: an ordinarily exercised function can contain untested combinations of its predicates. This is the limited sense in which self-generated tests "missed" the bug. No evidence supports the stronger assertion that generated testing inherently conceals defects or that the present grid now covers every meaningful path.

Table 6: Custom AST statement-line reach inside the candidate `verify()` only; neither MC/DC nor full test adequacy.

Audit stage	Count
240 generated cases; 1,200 item calls	40 / 53
Generated cases plus targeted probes	49 / 53
Newly reached statement lines	9
MC/DC achieved	Not measured
Mutation score	Not measured

Source-line instrumentation does not establish the correctness of the conditional expressions. A line with seven conjunctions can be executed even if the test never isolates the failure of each clause. The factorial grid changes each clause, which is stronger for these ten explicitly chosen fields, but it is still bounded by the

author-defined domain and oracle. Independent review of the specification, plus operator-level mutants or formally derived conditions, is required to characterize test adequacy more fully.

7.4 Multi-record order permutations

Across six two-notice patterns, sixty deterministic shuffles per pattern produced no eligibility changes. This supports a narrow statement: within those records and permutations, the revised verifier's Boolean result was insensitive to source-list order. It does not establish ordering invariance of citations, explanations or all forms of real chronological updates. In particular, equal-day conflicts and a reinstatement after cancellation require event semantics more detailed than the current flat record list. The audit did not simulate simultaneous edits or database transaction isolation.

7.5 What the full generated set did not show

The historical and revised verifiers returned the same Boolean for each of the 240 generated imaging items and for all 1,200 tracking-item calls. That zero-change result is important, because it limits the claim that the correction improved the generator benchmark. The unchanged non-imaging counts largely come from programmed index schedules, not a substantive evaluation of clinical-document agreement. The grid exposes differences only after constructing combinations outside that ordinary generator. In other words, the reported discovery is about a missing test condition rather than improved measured performance on the original dataset.

7.6 Existing renderer and citation checks

The earlier deterministic citation audit reports 736 distinct source identifiers encountered across 240 per-case template summaries: 15 proof source IDs for qualifying imaging cases and 721 inspection references for non-qualifying or rejected records. The IDs embed the case number, so they are globally distinct within the archived output. These values do not represent 736 independently useful medical citations. Matching the renderer's proof list to the verifier's own returned list mostly tests internal consistency, and a valid inspection locator establishes that an ID exists, not that its clinical interpretation is correct. The imaging-centric audit does not certify all care-coordination views, SOAP note paths or generated-answer routes.

A previously identified integration concern involved source IDs being displayed in the wrong citation category and some live-answer paths losing visible citation markers. The package's local checks cannot prove the hosted route is fixed. This risk remains open pending an integration test against the actual deployed service, with exact route output and version hash retained. The manuscript does not treat the public GitHub application's presence as proof of a corrected import or revised hosted execution.

8 Conservative default policy and executable tests

8.1 A policy change rather than an inferred clinical truth

The corrected identity-and-chronology predicate remains valuable for examining a specific implementation mistake, but it is not a safe default simply because it produces more positive documentary states. The present *research package* therefore names it `claimslens_rule_candidate.py` and places a conservative decision facade in `claimslens_verifier.py`. The facade first obtains the candidate verdict at an explicitly supplied `as_of` date. It then checks every source associated with the same tracking-item identifier for a status notice. Recognized events include `CANCELLED`, `REVOKED`, `ENTERED_IN_ERROR`, `AMENDED`, `CORRECTED` and `SUPERSEDED`. Even if a notice lacks a signature, mismatches a patient or refers to another modality, its presence is treated as a *data-quality conflict to review*, not as a credible veto or an affirmative clearance.

The resulting three states are DOCUMENT_FOUND, REVIEW_MISSING and REVIEW_CONFLICT. The first means a developer-defined candidate check found qualifying structured documentary evidence and the facade detected no linked status notice. Missing means no qualifying completion-like record was established; Conflict includes attached notices, a completion-like but nonqualifying source, or malformed input. The return object separates accepted support IDs from IDs merely inspected and carries explicit reasons and the clock date. None of these names means *result clinically reviewed*, *care complete*, or *safe to close an insurance claim*.

8.2 The safety–workload trade-off is visible

A wrong-patient cancellation attached to the correct tracking-item pointer was a false-open input under the historical Boolean relative to the developer's narrow candidate specification. The candidate correction ignores that notice and returns eligible. The conservative default returns REVIEW_CONFLICT because the source association itself is suspicious. These are different answers to different requirements: source identity controls whether a notice can *authoritatively veto* a report; source association controls whether a notice should be *surfaced for review*. They should not be represented as competing empirical clinical-truth labels.

The same conservative decision applies to an earlier matching cancellation, an unsigned notice, an unrecognized authority or an entered-in-error notice. It is intentionally possible for this policy to increase unnecessary reviews and duplicate follow-up. The present generated set has no independently labeled workflow outcome, so we cannot calculate false-positive rates or quantify burden. In the complete 1,200-item catalog, the conservative facade returns 283 DOCUMENT_FOUND, 782 REVIEW_MISSING and 135 REVIEW_CONFLICT states. The first number equals the candidate's programmed eligible count because none of the generated affirmative cases contains the adversarial attached notice at issue. These are program states, not patient outcomes.

8.3 Automated policy tests, coverage, and seeded faults

The packaged pytest suite contains **51 passing parameterized cases across 12 test functions**. It changes cancellation identity, kind, modality, laterality, source authority, signature, status-event type, date, invalid/missing completion fields and record ordering; it also checks empty evidence, a future-dated report, malformed source data and a required explicit audit date. The tests show the facade's software behavior on developer-written requirements. They are more informative about the default policy than repeated generator agreement but remain dependent on the author's specifications.

Using Coverage.py with branch measurement, `source/claimslens_verifier.py` executes **40/43 statements (93%)** and **4/6 branches (67%)**; the combined metric is **90%**. Unexecuted lines 37 and 41 raise errors for `as_of=None` and a non-date/non-ISO value; line 85 is the convenience `evidence()` wrapper. The two unvisited branch exits lead to those input-validation errors (36→37 and 40→41). The metric does not describe the large inherited candidate, user interface, intake route, model extraction or clinical decision process. It must not be conflated with the separate custom AST figure of 40/53 → 49/53 measured for the identity-aware candidate's function.

A transparent seeded-fault experiment edits five sites in temporary copies of the facade: removing all notices, requiring signature before review, overlooking patient mismatch, ignoring future-dated notices and suppressing the notice conflict condition. The same policy tests fail under all five altered versions. This is *five hand-selected faults detected out of five tested*, not a tool-generated comprehensive mutation score and not evidence of clinical validity. A full, externally reviewed code-mutation campaign remains a planned test.

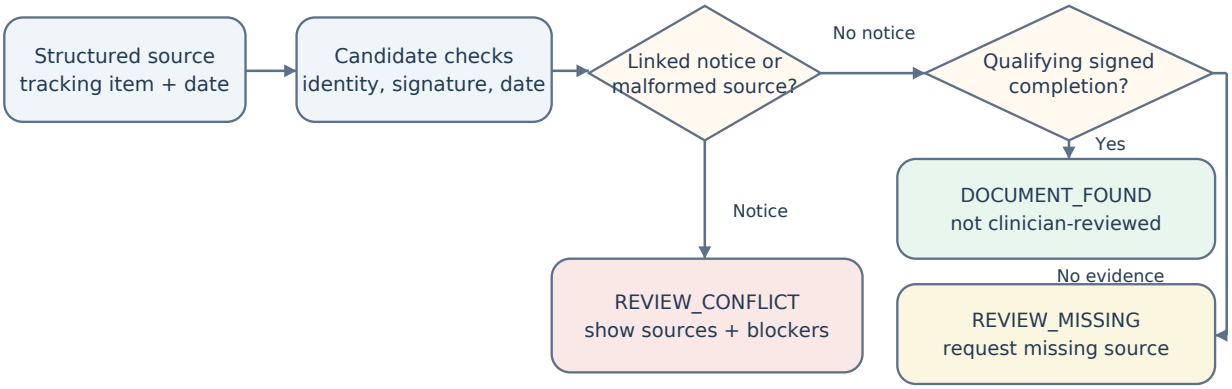


Figure 3: Research-default triage facade. It reads the prior identity-aware candidate outcome but withholds automatic documentary affirmation when an attached status notice or malformed source may require human reconciliation. The conservative decision is not shown to be the policy running in the hosted demonstration.

Table 7: Current research-default outcomes by tracking-item kind, tested at 17 October 2026. Counts are determined by the generated cases and developer policy, not clinician labels.

Tracking item	Document found	Review missing	Review conflict	Why these numbers are not performance metrics
Imaging	15	90	135	Generated report/error scenarios
Consultation	80	160	0	Index modulo three
Authorization	60	180	0	Index modulo four
Therapy	48	192	0	Index modulo five
Work status	80	160	0	Index modulo three
All 1,200 tracking items	283	782	135	No independent ground-truth adjudication

Table 8: Local executable checks on the conservative research facade.

Test or measurement	Outcome
Parameterized pytest cases (12 functions)	51 passed
Measured combined line/branch coverage	90%
Seeded source faults detected	5 / 5
Generator cases evaluated	240
Conservative policy on live hosted demo	Not checked
Independent medical adjudication	Not performed

8.4 Boundaries and residual defects

The default facade is a hold-and-triage policy, not an ingestion or reconciliation system. Unknown event names, incorrect tracking pointers, mislabeled status fields, duplicate IDs, amendment chains, effective-time semantics and false conflict volume need independent specifications. When conflicting notices are common, blanket holding could cause excessive alerts and delayed care coordination. A treating clinician, claims representative and nurse case manager would have to agree on action ownership and acceptable risk before using the policy in practice. It is distributed for inspection and further testing, not installed into the public demo by this report.

9 Asymmetric safety and the conservative research default

9.1 A false document label is not symmetric with a false-open label

The historical false-open can cause repeat requests, delays or a misleading suggestion that a complete report is missing. The opposite error—displaying an affirmative documentary label when an unresolved notice exists—could lead a user to stop seeking evidence or to mistake the status for completed clinical review. The consequences depend on the task and the reviewer. This study has no measured harm rates or usability observations, so it cannot assign an empirical probability or a validated risk-ranking. Nevertheless, the possibility that a false affirmative is more consequential than rework creates a reason to evaluate the two error directions separately rather than optimizing overall agreement with generated labels.

The corrected rule intentionally ignores an earlier matching cancellation when a later report exists. One interpretation is that the later report demonstrates an examination occurred and supersedes the cancellation. Another is that the order was cancelled and no one has documented reinstatement, making a subsequent report potentially linked to the wrong order or service. Both interpretations are coherent descriptions of different operating policies. The permissive candidate implements the first interpretation; the conservative research default implements neither interpretation automatically and holds the item for review. The necessary clinical or administrative policy decision was not collected from nurses, ordering providers, utilization reviewers or legal advisers. The program should not silently assert that its choice is a standard of care.

Likewise, an unsigned or wrong-authority notice is ignored by the current authenticated-cancellation veto. That helps prevent anonymous or forged messages from automatically removing a verified report. But silence is not the only possible response. A suspicious notice can still be material to data quality, security investigation or human reconciliation even when it lacks authority to change the final status. A nuanced interface may present DOCUMENT FOUND and a separate nonblocking integrity alert; alternatively, a conservative workflow may require manual review before displaying an affirmative label. The present evidence cannot select between them. Both require explicit requirements and outcome measurements.

Table 9 shows the default research facade on eight constructed notice patterns, alongside the permissive candidate. These are observed software outputs, not independent clinical judgments.

The conservative default may increase unnecessary reviews; without independent labels or users, these outputs establish neither clinical benefit nor acceptable workload.

9.2 Hazard identification with explicit evidence levels

Table 10 identifies ten groups of hazardous situations. It separates cause, potential harm and the strongest evidence status for controls. The classification is *preliminary, developer-authored*: there are no measured probabilities, probability bins, organizational acceptance criteria or residual-risk signoffs. Calling a risk “rare” on invented data would be misleading. A later ISO 14971-style process should obtain real operational observations, define acceptability before testing and evaluate each control with experts. A screening severity ranking is not a compliance claim. The table states consequences qualitatively rather than inventing empirical severity rankings or event probabilities. A provisional harm-priority assessment marks an affirmative status without the required evidence as potentially high consequence, while unnecessary review mainly creates delay and burden; these judgments require stakeholder validation.

The most direct defect evidence concerns H2: a specific constructed record made the earlier rule withhold

Table 9: Conservative default facade outcomes on eight constructed scenarios; not the hosted demo status.

Attached notice	Candidate eligibility	Default research status
No notice with signed qualifying report	Eligible	DOCUMENT_FOUND
Matching cancellation preceding report	Eligible	REVIEW_CONFLICT
Matching cancellation after report	Ineligible	REVIEW_CONFLICT
Matching future-dated cancellation	Ineligible	REVIEW_CONFLICT
Unsigned notice on same tracking item	Eligible	REVIEW_CONFLICT
Wrong-authority notice on same tracking item	Eligible	REVIEW_CONFLICT
Wrong-patient notice carrying same item pointer	Eligible	REVIEW_CONFLICT
Wrong service kind on same tracking item	Eligible	REVIEW_CONFLICT

an otherwise valid report according to the developer's fixed requirements. For H1, H7 and H8, the critical issue is not whether the code matches its specification but whether the specification itself is clinically or operationally acceptable. Treating developer opinion as risk control validation would reproduce the same oracle problem that originally motivated the paper.

9.3 Provisional safety assumptions and lifecycle status

If ClaimsLens were intended to inform clinical follow-up decisions without effective independent controls, the consequences of an erroneous affirmative could be serious. One could therefore provisionally assess the design against an IEC 62304 high-harm scenario, rather than assume low class because a nurse might review the output. This is *a screening assumption, not an IEC software safety-class determination*: classification requires a fixed intended use, harm assessment, qualified review, and documented effectiveness of external risk controls [12, 15]. Because the present product's intended use spans administrative tracking and possible medical interpretation, even the relevant regulated software function needs to be decided before a formal class can be defended.

The V-model in Figure 4 shows that unit/predicate verification exists while user-needs validation and live route testing are missing. It draws on the distinction explained in Papademetris and colleagues' Chapter 1, not on an invented claim about the contents of uninspected later textbook chapters [20]. Clinical usability research should be developed with IEC 62366-1 principles; organization-wide quality management would require a separate ISO 13485 process [13, 14]. None is represented here as achieved certification.

10 Language-model component: archived comparison, not efficacy

10.1 What the 64-case archive preserves

The historical model experiment used Qwen 2.5 3B through local inference on a balanced set of four cases for each of sixteen generator scenarios. The total was 64, with four positive cases and 60 negatives. The available aggregate records report an ungated source-context arm with three true positives, one false negative, eleven false positives and forty-nine true negatives; the same source-context proposals filtered by a deterministic gate gave three true positives, one false negative, zero false positives and sixty true negatives. These are reconstructed archived counts and do not constitute a new model execution in this report.

The ungated arm is important because it is the only preserved sign that the model could propose completion

Table 10: Preliminary hazard inventory, *not* an ISO 14971 risk assessment. Probability, acceptability and residual risk remain unestablished.

ID	Hazardous situation	Potential harm / failure direction	Control evidence and gap
H1	Affirmative label despite material contradictory document	Reviewer could wrongly treat report evidence as settled	Conservative default flags any attached status notice (51 parameterized pytest cases); clinical policy unvalidated
H2	Incorrectly missing final record due to unrelated cancellation	Duplicate request, delay, administrative burden	Historical bug reproduced; revised exact-ID predicate passes constructed tests
H3	Wrong-person or wrong-order linking upstream	False support or false-open, privacy exposure	Exact-ID comparison tested; intake and person matching not evaluated
H4	"Document found" read as "clinician reviewed"	Review or follow-up omitted because label overread	Wording explained in paper; no nurse comprehension study
H5	Renderer/source route drops locator or mixes rejected/proof IDs	User may not inspect basis for an output	Local template audit; hosted routing has no retained passing artifact
H6	Unreviewed risk score or automatic clinical interpretation	Poor prioritization or unsupported clinical inference	No predictive validity, calibration or user impact study
H7	Completion after cancellation without reinstatement proof	Potential false document-found result	Permissive research comparator accepts earlier notice; conservative default flags it; clinical policy unresolved
H8	Unsigned / wrong-authority cancellation ignored silently	Potentially relevant integrity warning suppressed	Conservative default flags attached notices; no workload or effectiveness measurement
H9	Amended, stale, OCR-corrupted or merged documents	Inaccurate linkage and late-stage propagation of error	No ingestion/OCR/amendment study; proposed in future work
H10	Unauthorized exposure, alert fatigue, drift, audit-log tampering	Privacy harm, missed warning, unverifiable action	No deployment security or prospective human evaluation in this report

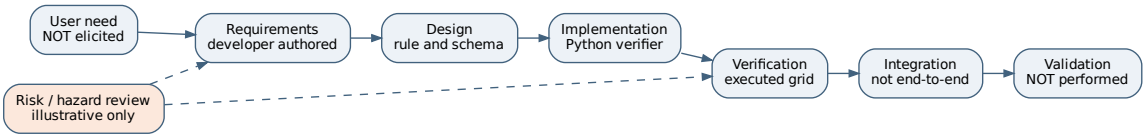


Figure 4: Evidence-status adaptation of the V-model: the study has an implementation and executed code checks, but no elicited user requirements, independent clinical validation or passed deployed-route integration study. Dashed risk links represent planned work, not verified safety controls.

unsupported by the verifier. The gate removed eleven such false positives in the archived aggregates. But the gate could only veto, never add a true positive absent from the verifier, so the experiment was mathematically unable to demonstrate added documentary sensitivity over an already generator-matched verifier. With only four positives in 64 cases, the always-negative baseline already scores 60/64 (93.75%) accuracy; gated accuracy is therefore uninformative about extraction capability or real-world utility. The unstructured-text, independently labeled field-extraction task proposed in Section 13 (Study C) replaces this Boolean setup for a meaningful medical-AI evaluation.

The full prompts, model digest, per-case outputs, malformed-output handling and token truncation logs were not retained as an auditable record. Although earlier notes mention generation settings, this manuscript does not treat them as proven exact experimental configuration for a reproducible run. It cannot distinguish model reasoning errors from prompt construction, parse failure, truncation, or variation in the reference implementation. Confidence intervals on these template-clustered cases would not quantify a well-defined target population. The right conclusion is that the archive il-

Table 11: Legacy diagnostic artifact: archived Qwen model comparison, not reproducible at case level or a measure of text-extraction ability. Positive denotes a generator-defined qualifying imaging record.

Arm	TP	FN	FP	TN
Always negative	0	4	0	60
Qwen, source context, ungated	3	1	11	49
Qwen, source context, gated	3	1	0	60
Verifier alone on generated labels	4	0	0	60

lustrates how a veto can suppress unsupported model proposals in a toy setting; it supplies no independent evidence of improved clinical extraction, nurse efficiency or end-to-end reliability.

10.2 The experiment an AI paper would actually need

A useful future task is *field-level extraction* from genuine or independently written report text. The model would identify patient/order identifiers when present, modality, anatomy, side, performed/report dates, signature/authentication language, report status and amendment or cancellation references. It should quote the verbatim span supporting each extracted value. The verifier would be run unchanged on normalized fields, and errors would be attributed separately to extraction, source identity resolution and deterministic decisions. Such a design can show whether the model helps convert unstructured evidence into auditable fields; it is not guaranteed to produce any model benefit.

The included `benchmark_extraction_scaffold.py` accepts independent-label and prediction JSONL files and computes field-level exact agreement, abstentions and missing source-span counts. No real or independent labels are shipped, so no extraction accuracy is reported. This is an evaluation interface, not a completed benchmark. The future protocol will need both a non-LLM baseline and a controlled model comparison, constrained output, model version identifiers, case-level logs and disagreement adjudication. It should also test indirect prompt injection inserted into report text, where the model must not obey instructions that arrive as untrusted medical source data [9, 19].

11 Interoperability, privacy, and deployment boundaries

11.1 FHIR provides an explicit language for report state

FHIR R4 represents diagnostic reports as `DiagnosticReport` resources with an explicit `status`, while related workflows can involve `ServiceRequest`, `Appointment` and `Provenance` [10]. A final report can subsequently be amended, corrected, cancelled or entered in error. The ClaimsLens local records use a simpler `event`, `superseded` and date model; they do not implement a lossless FHIR mapping. A proposed mapping would associate imaging orders with `ServiceRequest`, encounter scheduling with `Appointment`, report results with `DiagnosticReport`, and author/timestamp information with `Provenance`. It must preserve document version history rather than flatten all states into one Boolean.

Table 12: Conceptual alignment only; no FHIR serialization or round-trip test was run.

Prototype concept	Candidate FHIR R4 concept
Imaging request	ServiceRequest
Appointment / booking	Appointment
Signed final imaging report	DiagnosticReport with status and performer
Author and source lineage	Provenance
Amended/corrected report	DiagnosticReport version/status + Provenance
Clinician acknowledgment	Explicit workflow documentation; not inferred from report status

This mapping is a design hypothesis, not a validated interoperability implementation. It cannot resolve

inconsistent identity, duplicated records, nonstandard faxes, missing timestamp granularity or the institutional meaning of a cancelled order. An evaluated FHIR exercise should generate bundles from an independent tool such as Synthea, preserve source provenance and compare round-trip outcomes against independently annotated expectations [26]. The case study has not performed that exercise.

11.2 Administrative support versus regulated decision support

The FDA's January 2026 Clinical Decision Support Software guidance describes statutory exclusions and the interpretation of certain software functions [7]. ClaimsLens has an administrative-document-tracking function, an optional language-model component and unvalidated risk-scoring ideas. They must be assessed as distinct functions before anyone concludes that a particular exemption or device framework applies. Describing a task as "documentation support" is not a categorical exemption, and adding clinical triage or automated recommendations may change the analysis. The paper therefore does not claim FDA review, clearance, non-device status, or legal compliance.

The HIPAA Privacy Rule has specific provisions for workers' compensation disclosures. HHS explains that workers' compensation insurers, administrators and employers are not automatically covered entities merely because they handle compensation matters; providers supplying the information may be covered, and the applicable state law matters [11]. This is why a blanket assertion that every carrier or third-party administrator (TPA) has the same HIPAA duties would be inaccurate. The project has no actual patient records, but a live version would require entity-specific privacy analysis, access controls, audit retention and a defensible minimum-necessary workflow.

11.3 The public repository is not the audit release

The public source at `https://github.com/sohaarian/claimslens-public-demo` establishes a visible demo repository, including `claimslens_final.py`, `claimslens_public_demo.py`, and other application modules. It is not, at the time of this report, a tagged public release of the independently packaged research audit and its canonical `claimslens_verifier.py`. The public code link supplies product context only; the accompanying archive must be preserved separately for reproducibility. No DOI, release tag, public research commit or verified live deployment version is claimed.

Repository traceability warning. *The public demo at `https://github.com/sohaarian/claimslens-public-demo` is an illustrative user-interface prototype. The authoritative, checksum-verified artifacts for this study are contained solely in the accompanying research-package archive, not the public demo repository.*

The report's SHA-256 checksum identifies the exact research verifier file in the accompanying archive. It does not prove the public app runs that file. End-to-end verification would require inspecting the deployment's actual import graph, capturing the running commit and image digest, exercising the relevant API/UI route, and retaining the resulting source IDs and status. Until that is done, the deployment-to-audit equivalence question stays open rather than being quietly folded into an experimental software result.

12 Threats to validity and alternative explanations

12.1 Specification, labels and author dependence

The principal internal-validity threat is shared assumptions. The same developer defined the original records,

encoded the expected labels, inspected the verifier, wrote the alternative predicate and constructed the test grid. Even a separately coded oracle can repeat the author's misunderstanding of a real cancellation policy. For that reason this report uses the phrase *developer-oracle agreement* rather than correctness. An independent clinical, utilization-review or information-management adjudicator would need to determine whether cancelling an imaging order should alter a status after a performed study, and how order reinstatement, amended reports and signed but future-dated notices should be represented.

The case `CL-0001` is useful because it contains a qualifying final report. It is not representative of a clinical population. Expanding a single case into thousands of predicate variations increases confidence about the coded Boolean relationships but does not increase the number of independently observed encounters. We explicitly do not report a confidence interval treating those grid cells as independent patients. If an independently authored clinical dataset were available, uncertainty should reflect the unit of annotation, site, patient and report clustering, and any intended sampling population.

12.2 Construct and ecological validity

The variable called documentary eligibility is constructed from exact-ID, date and signature fields. It does not directly measure a clinician reading a result, a safe follow-up, the occurrence of an examination, or time saved by a nurse. A correct report could have wrong structured metadata; an incorrect report could appear structurally complete. Thus the construct is best understood as a programmer-defined document predicate. Mapping it to patient safety or workers' compensation outcomes would require additional evidence and professional involvement.

The synthetic generator does not contain realistic distribution of age, language, injury severity, multimorbidity, regional practices or document transmission channels. It also lacks typical administrative complexities such as authorization modifications, jurisdictional forms, delayed specialty consultation, reserve decisions or litigation referral. These limits reduce ecological validity. The paper's title mentions medical-document evidence rather than promising validated workers' compensation operations. Workers' compensation is the motivating domain, not an observed evaluation setting.

12.3 Coverage and implementation validity

The 40/53 and 49/53 values depend on one source version and a particular instrumentation method. They are not comparable to an external coverage report for a different build or Python release without keeping the hash, interpreter version and instrumentation definition fixed. The added predicate grid includes ten field toggles, but test adequacy also concerns decisions involving multiple accepted reports, event priority, duplicate IDs, null fields, route rendering, and combinations not represented in the grid. The study performs a fixed collection of 360 permutations, not exhaustive ordering over arbitrarily many records.

One error class in the current rules is conceptual rather than syntactic. A later completion is accepted after an earlier matching cancellation unless another disqualifying record exists. That policy may be correct under some definitions and incorrect under others. There is no purely software-based way to determine its clinical acceptability from the existing fixtures. The conservative research facade deliberately chooses a review-first default. The research finding is the disagreement between two plausible policies; safety requires an external requirement, not a larger test grid alone.

12.4 Security and deployment validity

The predicate audit accepts structured objects directly and bypasses any upload pipeline, OCR component or human-edited case attachment. It therefore does not show that malicious or accidental mislinking occurs, nor that the system safely rejects malicious file payloads.

It also cannot validate authentication, authorization, session controls, rate limits, data isolation or retention policies in the public app. Prior implementation work on such controls should be assessed with its own technical evidence rather than used to infer that the documentary verifier has passed end-to-end assurance.

The public demo and the research package have separate identifiers and histories. Without a deployed commit or container digest and an end-to-end replay of the exact problematic input, the hosted status behavior remains unknown. This is particularly important because earlier public source files and the research-corrected verifier share related names but not necessarily identical logic. A release traceability table should record *research version*, *public repository commit*, *deployment identifier* and *verified route*, with none silently substituted for another.

12.5 AI and statistical validity

The archived Qwen set has only four positives. The model was constrained by a gate that could veto an unsupported proposed positive but not contribute a positive the verifier rejected. Therefore the design cannot identify incremental extraction value over a perfectly matched generator-rule baseline. Its missing prompts and outputs also make exact replication impossible. These are not problems repaired by reporting more decimal places or Wilson confidence intervals. They require a new task definition and new evidence. Future analysis should use independent report annotations, complete logs, source-span verification and prespecified error attribution.

13 Prioritized future studies with decision criteria

The study sequence is chosen to reduce the most consequential uncertainties first. In particular, a clinical labeling study should precede any claim that an alternative cancellation policy is safer; a software mutation study should precede a claim of adequate test strength; and independent extraction evaluation should precede any medical-AI efficacy statement. No results below are represented as completed. Each protocol specifies a bounded input, comparison, outcome and possible stop decision.

13.1 Study A: independent requirements and cancellation adjudication

Objective. Establish whether the statuses and document transitions defined by the developer match the intended users' needs and clinical responsibilities. Recruit three to five workers' compensation nurse case managers for short contextual interviews and at least two independent subject-matter annotators, ideally including a clinician familiar with imaging-result management. The small interview group is an exploratory sample, not a prevalence or effectiveness study. Participants should be asked to describe actual document handoffs, missing records, ambiguity in order identifiers, escalation expectations and the distinction between report available, report reviewed, and care completed.

Material. Create an adjudication packet of at least 40 independently selected narratives balanced across missing evidence, mismatched patient, wrong order, cancellation before/after performance, same-day ambiguity, later report without reinstatement, unsigned notice, amended report and clinician acknowledgment. The cases should be written or approved by reviewers who have not seen the existing verifier Boolean. The precise count is a feasibility target; sample-size justification will depend on pilot disagreement rates and adjudication resources. Include source timelines and documents, not only developer-created labels.

Procedure. Have annotators independently classify each case into DOCUMENT FOUND, REVIEW MISSING, REVIEW CONFLICT and NOT DETERMINABLE, recording the evidence span and the reason for each choice. Lock the initial decisions before adjudication. Record unresolvable disagreements instead

of rewriting the ground truth to match software. Compare the historical, revised and conservative overlay outputs only after the independent annotations are finalized. The outputs are raw agreement matrices, reasons for disagreement and a proposed revised requirement specification.

Criterion and decision. The principal gate is whether domain experts agree on a policy for reinstatement, ambiguous dates, wrong-authority messages and acknowledgment responsibility. Do not ship affirmative status automation if material cases remain unresolvable or if the reviewed clinical policy contradicts the code. An observed zero-disagreement result in a small sample would not prove zero clinical risk; it would authorize a broader study with a documented risk-based sampling plan.

13.2 Study B: testing adequacy beyond generated labels

Objective. Evaluate how sensitive the suite is to actual code defects, not merely whether it agrees with the current specification. Construct a versioned software requirements specification using explicit *shall* statements for identity matching, authority, timestamp ordering, signature, supersession, event precedence, rejected-source citation and user-facing status. Each rule should have positive and negative tests, with independent review of its expected outputs.

Procedure. First use a property-based generator to vary each required field across valid, invalid, missing, malformed and conflicting values; preserve seeds and minimum failing examples. Second define metamorphic relations: adding an irrelevant record should not silently change evidence eligibility under the approved policy; reordering an unordered source list should preserve both eligibility and evidence-set membership; adding a second identical source should not create a new clinical event; correcting a nondecisive typo should not change the status. Third implement code mutation testing of cancellation and report predicates. Explicit mutants include deleting the patient check, deleting the order check, reversing a date comparison, treating `signed` as optional, and ignoring `superseded`. Record which tests kill each non-equivalent mutant.

Independent implementation. Ask another programmer to implement the approved specification without viewing the developer-written Python specification. Compare both on a preregistered batch, retain disagreements and inspect ambiguous cases rather than silently modifying the tests. Branch and condition coverage should be measured with a reproducible coverage tool. The present statement-line instrument can serve as a baseline but must not be relabeled as MC/DC. A reasonable engineering entry gate is that every high-consequence requirement has a negative test and every deliberately seeded identity/date mutant is detected; the final mutation-score threshold should be set before testing and justified by feasibility, not invented retrospectively.

Reporting. Provide the exact mutant list, killed and surviving counts, excluded equivalent mutants, coverage denominator and source SHA. Separate defect discovery from bug regression. A second defect class, such as incorrect supersession priority or wrong-date handling, would be valuable if independently found, but no second defect is claimed in this report.

13.3 Study C: realistic medical-report extraction with an AI baseline

Objective. Determine whether a language model can accurately extract fields from medical-report text under a separate independently annotated task, compared with a non-LLM baseline. This is the highest-value route to an actual medical-AI contribution, because it tests a role not already solved by the deterministic verifier. The protocol should compare a transparent regex/rule baseline, a compact current model and a stronger contemporary LLM, with exact model revision and decoding configuration recorded.

Material. An initial pilot may use approximately 50–

100 legally usable and independently reviewed radiology narratives, but the sample size is provisional; a confidence interval precision or error-discovery rationale must be stated before data collection. Public datasets such as Indiana/Open-i or MIMIC-CXR predominantly address chest imaging and cannot be passed off as representative of lumbar, knee or upper-extremity workers' compensation studies. A separate clinician-written musculoskeletal synthetic collection can be used for development if its provenance and limitations are recorded. Do not place actual patient content in the public repository.

Labels and outputs. Independently annotate patient and order linkage when present, modality, anatomical site, laterality, report state, report date, amended/superseded status, signature authority and relevant source span. Measure field-level exact match, precision, recall, abstention, parsing failure, incorrect source-span rate, and downstream changes to verifier output. Keep false affirmative and false negative outcomes separate. A model must not invent an absent patient identifier simply to fill a schema. Each extracted value should be traceable to exact visible text and reviewed before being treated as an authenticated field.

Control of confounding. Use fixed prompts, JSON schema or constrained decoding when available, bounded character budgets, recorded truncation flags and more than one deterministic seed or replicated run where appropriate. Randomize source order when ordering is not meaningful and preserve medically significant chronology when it is. Evaluate cluster-aware uncertainty by report source and patient where multiple reports occur. Stratify results by modality, document status and input degradation. Challenge the model with prompt injections inside report body text and with OCR-like noise. The downstream verifier must not automatically accept model-generated identifiers as trusted records; source evidence and human verification remain required.

Decision rule. Establish before running whether added extraction benefit outweighs new false-positive, latency, privacy and reviewer-burden risks compared with the rules baseline. An LLM should not be kept merely because it gives a high raw accuracy on a small unrepresentative template set. The included JSONL scoring scaffold is ready for independently sourced labels but contains no evaluation results.

13.4 Study D: clinical workflow, governance and end-to-end safety

Objective. Assess the workflow as used by nurses and other authorized stakeholders, rather than only the Python rule. Prototype distinct Missing, Conflict and Document Found states and require reviewers to distinguish document existence from clinical acknowledgment. Run formative think-aloud sessions and observe whether users can find the deciding report, recognize contradicting notices and override a mistaken status. After usability revisions, compare groups shown a terse label, a label with evidence, and a deliberately incorrect seeded affirmative. Measure error detection, time to decision, unnecessary follow-up and confidence calibration; do not use real patients in a novice study.

Interoperability and intake. Design a bounded FHIR R4 import/export test with simulated ServiceRequest, DiagnosticReport, Appointment and Provenance resources. Run round trips on Synthea-derived bundles only after checking the resource mapping. Independently challenge patient matching with missing identifiers, formatting differences and realistic OCR degradation. Test report amendments, corrected statuses, same-day timestamps, late data arrival and revoked orders. Keep the ingestion layer separate from the verified core so route, parsing and identity failures can be attributed.

Risk governance. Complete a documented hazard analysis with stakeholder-approved harm definitions, likelihood or conservative worst-case assumptions, risk acceptability criteria, verified controls and residual-risk decisions. Identify the actual legal entity and data flow before drawing conclusions about HIPAA applicability.

Review access control, audit-log integrity, retention, incident response, drift monitoring and the possible impact of an automated risk score. The lifecycle plan should include software versions, data/model change control, test traceability and withdrawal/decommissioning plans. Only after those steps should a provisional IEC 62304 classification and regulatory determination be finalized.

13.5 Why the order matters

A model-extraction study without trusted ground truth may create another circular accuracy number. A broad rule-grid study without independently confirmed clinical semantics may produce thousands of passing but unhelpful tests. A nurse study without a stable interface and documented source locators may confound poor usability with incorrect underlying data. The sequence A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D is intended to close those dependencies while acknowledging that some activities, such as privacy review and threat modeling, should begin in parallel. Early negative findings should be published or documented rather than discarded.

14 Implementation recommendations

For a developer preserving the current prototype, the immediate recommendation is to keep the conservative research API and the permissive rule comparator separate. Document status outcomes should include a machine-readable reason and an evidence ID set. The user-facing label should never say "care complete" or "clinician reviewed" unless a separate source explicitly establishes that event. A cancellation attached to an unrelated patient should trigger at least a detectable integrity condition somewhere in the ingest-and-review workflow, even if it lacks authority to veto a qualifying report. This is a recommendation for a requirement discussion, not a claim that the policy has already passed acceptance testing.

For a research reviewer, the strongest reproducibility check is to run the complete audit command with no network access, inspect the generated JSON and compare the verifier SHA. The grid can be rerun in seconds to minutes and does not require a GPU, Ollama or access to a real claim. It should be followed by code review of the reference specification and the mutation sites. More audit cases alone will not turn a developer-authored Boolean into a medical gold standard.

For a potential institutional partner, the next resource should be access to appropriate subject-matter experts and permissible report text, not immediate deployment into a claims system. One nurse manager, a clinician and an information-security reviewer could first challenge the state definitions and failure directions; those consultations would materially change the value of subsequent engineering tests. Until that happens, the prototype is more responsibly described as a research system for evidence-integrity questions.

15 Audit interpretability and operational decision records

15.1 Why reasons must accompany a Boolean

A Boolean is appealing for automated workflow, but it cannot explain whether the missing evidence is absent, present with a mismatching patient identifier, a scheduled appointment mistaken for a completed exam, or a valid report contested by a cancellation notice. The current verifier returns a status, a set of blockers, traces of failed checks and citation identifiers. Those fields can be evaluated separately. A program may compute the correct Boolean while presenting the wrong reason or an unrelated source to the human user. Conversely, a reviewer might act incorrectly despite a correct evidence trail. This is why the risk analysis keeps decision quality and explanation integrity separate.

For reproducibility, a decision log should retain the evaluated claim and tracking-item identifiers, the `as_of` timestamp, the exact verifier version, the source record IDs actually considered, each decisive predicate and the

final source-rendering state. A clinical or administrative user interface should not show a source as affirmative proof merely because the source was inspected; that source may have been rejected. The distinction was already relevant to a local prototype route where inspection and support citations were not consistently separated. The new deterministic audits can find some ID-level mismatches, but end-to-end acceptance must verify what the deployed reviewer sees.

15.2 Recommended minimal decision log

A future logging schema can use eight fields: *request ID*, *case/item*, *input version*, *event and decision timestamps*, *candidate IDs*, *accepted IDs*, *rejected IDs with reasons*, and *final state plus reviewer action*. Event and decision timestamps should be retained in ISO-8601 UTC form (preferably millisecond precision, or at least minute precision where actually available), together with the precision and original timezone of each source. This permits ordering a report and cancellation that occur on the same calendar day when reliable time-of-day evidence exists; a date-only source remains chronologically ambiguous. Sensitive identifiers should be pseudonymized or protected under a documented access policy. Input changes should produce an auditable event with time and responsible actor, rather than silently rewriting an earlier evaluation. Such logging is particularly valuable when a late report, corrected document or contested cancellation changes the display days after an earlier decision.

An ordinary administrative record should not claim to prove that an ordering clinician received a result. If clinical acknowledgment is a future tracked event, it needs its own actor, time, source and evidence policy. Likewise, the human-review button should capture who approved what, under which status definition, and whether that approval applied to the data record, the suggested summary or an action in the claims system. Combining those events in one `closed` flag would undermine the fundamental distinction the research is trying to preserve.

15.3 Decision thresholds that cannot be inferred from the current corpus

The generated labels do not specify acceptable false-affirmative rates, allowable review workload, critical-case prevalence, or escalation timeliness. None can be estimated from the 18,432 grid cells. A design team should set thresholds prospectively with clinical and operational users and revisit them after testing under changed document mixes. A rule producing more conflict reviews may reduce one kind of risk while imposing delays and notification fatigue. A rule suppressing unsigned notices may prevent false vetoes while hiding useful warnings. The correct policy is therefore not the rule with the most passing test cases; it is the policy whose harms, work practices and verified controls are acceptable for a defined use.

16 Conclusion

A cancellation record associated with the right tracking pointer but the wrong identity could veto a signed imaging report under the historical ClaimsLens rule. The developer found the overbroad check during source inspection, constructed a case to reproduce it, and built a revised predicate. In the new enumerated software study, the revision removed vetoes in many deliberately mismatched record combinations while preserving rejection for matching later or ambiguous notices. The new results show better conformity to the selected identity-and-chronology rule, not stronger clinical safety. The revision was purely permissive across the 18,432-case predicate grid, and it still accepts a report after an earlier matching cancellation without reinstatement evidence.

The methodological lesson is specific. Agreement with program-generated labels is not an independent oracle. Coverage of most statement lines is not coverage of all conditions. A successful rule-checker is not evidence that a user would interpret its labels correctly. Likewise, an archived four-positive language-model run cannot establish added AI value when its

gate can only reproduce or reduce the verifier's already-perfect generated-label outcome. These are limits of the evidence, not failures that can be repaired through more confident wording.

ClaimsLens now has reproducible comparison audits and a conservative default research policy whose behavior is visible in the packaged outputs. Its medical AI question remains open until independently reviewed documents and source-grounded extraction are evaluated. The next meaningful contribution is not another claim of synthetic "accuracy," but an external specification and a study in which both software errors and human consequences can actually be measured.

References

[1] Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering, 41*(5), 507–525. https://doi.org/10.1109/TSE.2014.2372785.

[2] Callen, J. L., Westbrook, J. I., Georgiou, A., & Li, J. (2012). Failure to follow-up test results for ambulatory patients: A systematic review. *Journal of General Internal Medicine, 27*, 1334–1348. https://doi.org/10.1007/s11606-011-1949-5.

[3] Chen, T. Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T. H., & Zhou, Z. Q. (2018). Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys, 51*(1), Article 4. https://doi.org/10.1145/3143561.

[4] Claessen, K., & Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. *Proceedings of ICFP*, 268–279. https://doi.org/10.1145/357766.351266.

[5] Collins, G. S., Moons, K. G. M., Dhiman, P., Riley, R. D., Beam, A. L., et al. (2024). TRIPOD+AI statement: Updated guidance for reporting clinical prediction models that use regression or machine learning methods. *BMJ, 385*, e078378. https://doi.org/10.1136/bmj-2023-078378.

[6] Vasey, B., Nagendran, M., Campbell, B., Clifton, D. A., Collins, G. S., et al. (2022). Reporting guideline for the early-stage clinical evaluation of decision support systems driven by artificial intelligence: DECIDE-AI. *Nature Medicine, 28*, 924–933. https://doi.org/10.1038/s41591-022-01772-9.

[7] U.S. Food and Drug Administration. (2026, issued January 6 and reissued January 29). *Clinical decision support software: Guidance for industry and Food and Drug Administration staff*. https://www.fda.gov/regulatory-information/search-fda-guidance-documents/clinical-decision-support-software.

[8] Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM, 64*(12), 86–92. https://doi.org/10.1145/3458723.

[9] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv*, 2302.12173. https://arxiv.org/abs/2302.12173.

[10] Health Level Seven International. (2019). *FHIR Release 4: DiagnosticReport, ServiceRequest, Appointment, and Provenance specifications*. https://hl7.org/fhir/R4/.

[11] U.S. Department of Health and Human Services. (2003; current guidance page). *Disclosures for workers' compensation purposes (45 CFR 164.512(l))*. https://www.hhs.gov/hipaa/for-professionals/privacy/guidance/disclosures-workers-compensation/index.html.

[12] International Electrotechnical Commission. (2006, amended 2015). *IEC 62304:2006+A1:2015, Medical device software—Software life cycle processes*. IEC.

[13] International Electrotechnical Commission. (2015). *IEC 62366-1:2015, Medical devices—Application of usability engineering to medical devices*. IEC.

[14] International Organization for Standardization. (2016). *ISO 13485:2016, Medical devices—Quality manage-*

ment systems—Requirements for regulatory purposes. ISO.

[15] International Organization for Standardization. (2019). *ISO 14971:2019, Medical devices—Application of risk management to medical devices*. ISO.

[16] Jia, Y., & Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering, 37*(5), 649–678. https://doi.org/10.1109/TSE.2010.62.

[17] Kosny, A., Lifshen, M., Tonima, S., Yanar, B., Russell, E., MacEachen, E., et al. (2016). *Role of health-care providers in the workers' compensation system and the return-to-work process: Final report*. Institute for Work & Health. Institute for Work & Health report page.

[18] Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) Version 1.1* (NIST SP 800-218). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218.

[19] Tabassi, E. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* (NIST AI 100-1). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.100-1.

[20] Papademetris, X., Quraishi, A. N., & Licholai, G. P. (2022). *Introduction to medical software: Foundations for digital health, devices and diagnostics*. Cambridge University Press. https://www.cambridge.org/core/books/introduction-to-medical-software/.

[21] Pons, E., Braun, L. M. M., Hunink, M. G. M., & Kors, J. A. (2016). Natural language processing in radiology: A systematic review. *Radiology, 279*(2), 329–343. https://doi.org/10.1148/radiol.16142770.

[22] Russell, E., & Kosny, A. (2019). Communication and collaboration among return-to-work stakeholders. *Disability and Rehabilitation, 41*(22), 2630–2639. https://doi.org/10.1080/09638288.2018.1472815.

[23] Singh, H., Thomas, E. J., Mani, S., Sittig, D. F., Arora, H., Espadas, D., Khan, M. M., & Petersen, L. A. (2009). Timely follow-up of abnormal diagnostic imaging test results in an outpatient setting: Are electronic medical records achieving their potential? *Archives of Internal Medicine, 169*(17), 1578–1586. https://doi.org/10.1001/archinternmed.2009.263.

[24] Singh, H., Thomas, E. J., Sittig, D. F., Wilson, L., Espadas, D., Khan, M. M., & Petersen, L. A. (2010). Notification of abnormal lab test results in an electronic medical record: Do any safety concerns remain? *The American Journal of Medicine, 123*(3), 238–244. https://doi.org/10.1016/j.amjmed.2009.07.027.

[25] Gallifant, J., Afshar, M., Ameen, S., et al. (2025). The TRIPOD-LLM reporting guideline for studies using large language models. *Nature Medicine, 31*, 60–69. https://doi.org/10.1038/s41591-024-03425-5.

[26] Walonoski, J., Kramer, M., Nichols, J., Quina, A., Moesel, C., Hall, D., et al. (2018). Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association, 25*(3), 230–238. https://doi.org/10.1093/jamia/ocx079.

[27] Yanar, B., Kosny, A., & Lifshen, M. (2019). Perceived role and expectations of health care providers in return to work. *Journal of Occupational Rehabilitation, 29*, 212–221. https://doi.org/10.1007/s10926-018-9781-y.

Technical Appendix A: Audit reproduction

Purpose. This appendix is a user-independent technical artifact: the commands, file structure and expected result classes to rerun the finite software study. It does not replace the methodological interpretation in the report body. The archive is the artifact analyzed here, while the public demo repository is a related software demonstration with a separate provenance history.

A.1 Files required for deterministic tests

```
source/claimslens_verifier.py
source/claimslens_rule_candidate.py
source/claimslens_verifier_historical.py
source/audit_policy_generator.py
source/audit_seeded_faults.py
tests/test_conservative_policy.py
requirements-research.txt
source/run_research_audits.py
source/audit_factorial_grid.py
source/audit_conservative_policy.py
source/experimental_triage.py
source/check_field_mutations.py
source/check_all_obligations.py
source/check_cancellation_behavior.py
source/check_renderer_sources.py
source/benchmark_extraction_scaffold.py
results/audit_summary.json
results/factorial_grid.json
results/experimental_triage.json
results/policy_generator_results.json
results/seeded_fault_detection.json
results/policy_branch_coverage.json
SHA256SUMS.txt
README.md
```

The auxiliary application and studio modules included in the source folder exist to permit historical regression checks. The conservative research default is the `claimslens_verifier.py` file named above; it is not silently substituted for a public deployment module. The archive preserves a historical baseline separately. No network model downloads or clinical records are required to run these checks. The SHA-256 of the default research policy module is f9fe07e192a64fc08d1cc3ce5923385495a53cf6180c07494e5fa1e7b697fc34; the SHA-256 of the identity-aware permissive comparator is cd14284919dba346e03b6657a96ff7ce571ddce1bc6c5483e50478f0caf3463b. Both are local-file identities, not evidence that the hosted demo runs either version.

A.2 One-command audit and expected invariants

```
python source/run_research_audits.py
python source/audit_policy_generator.py
python source/audit_seeded_faults.py
python -m pytest -q tests/
python -m coverage run --branch --source=
    claimslens_verifier -m pytest -q tests/
python -m coverage report -m
sha256sum source/claimslens_verifier.py
```

The first command regenerates the case and cancellation audits and emits a summary. Expected assertions: 240 imaging fixtures; 1,200 tracked items; 283 generator-rule met and 917 open; zero changed results on original generated obligations; 18,432 factorial combinations; zero disagreements with the developer-coded oracle; 360 multi-source permutations without an eligibility change; and a future-dated matching cancellation not eligible for affirmative documentary status. The 40/53 and 49/53 measurements refer to counted executable statement lines in `verify()`, not to the whole project or MC/DC coverage. A fresh run is authoritative; a pasted older JSON is not.

A.3 Selected direct code excerpt

```
# Original cancellation condition, copied
# from historical verifier:
cancelled=any(
 r['event'] in ('CANCELLED','REVOKED')
 and r['authority']==authority
 and r['signed']
 for r in linked
)
# Current verifier validates identity,
# authority, signatures, service attributes
# and date ordering before veto/review.
```

The earlier test failure is located in the discrepancy between the report candidate's full checks and the old cancellation's shorter conjunction. The executed factorial test varies each of the ten declared cancellation properties, which provides more systematic predicate exercise than the initial hand-selected one-case probe. It still uses the author's definition of expected logic, and the paper never converts those matches into measured clinical accuracy.

A.4 Test-case date representation

The anchor report's completed date is calculated from the generator's exact case record. The cancellation is a copied report with the event changed. The six grid date categories include order-minus-one-day; report-minus-one-day; report day; report-plus-one-day; literal `2027-01-01`; and literal `not-a-date`. The archive records each concrete computed date in JSON. The audit clock is 2026-10-17. The grid also varies every cancellation identity/authority flag and three event values; no case-level output is omitted from the count calculation.

A.5 Disclosure of incomplete independent replication

The reported program outputs can be regenerated from the Python modules in the accompanying archive. A prospective collaborator should run the code from a clean environment and independently inspect the requirements. The package does not contain an expert gold-standard label set, an externally audited risk register, an Ollama reproduction of the old Qwen run, a deployed service response capture or an independent author attestation. It is intentionally possible to reproduce the *software* results without treating those omissions as solved.

Technical Appendix B: Design rules and traceability

B.1 Requirement-to-evidence map

Table 13: Explicit research requirements, test evidence and remaining validation. "Pass" means pass of developer-authored software assertion only.

ID	Preliminary development requirement	Executed evidence	Independent gap
R1	Only signed final report matching patient, claim, order and service proves documentary eligibility	240 generator checks; factorial toggles; default facade pytest cases	Clinician-approved definition missing
R2	Authenticated later matching cancellation prevents automatic eligibility	Factorial grid; single-case comparison; default facade pytest cases	Reinstatement semantics absent
R3	Same-day, future and unparseable matching dates require review	Factorial grid; fixed dates; default facade pytest cases	Real timestamp quality and notice meaning unknown
R4	Unrelated identities must not silently veto a valid completion	Identity toggles; hand probe; default facade pytest cases	Upstream mislinking policy untested
R5	Reordering records does not change Boolean eligibility for tested sets	Six patterns, 360 permutations	Explanation and citation order not validated
R6	A reviewable status includes source IDs and reasons	Local deterministic renderer check	Live SOAP/answer routes not confirmed
R7	Unauthenticated contradictory notices should be visible as a policy question	Conservative facade; 51 parameterized pytest cases	Requirement and benefit need independent review
R8	Report state and clinician acknowledgment remain distinct	Conceptual requirements analysis	Real nurse comprehension not measured

B.2 Output status vocabulary

The archived historical and permissive candidate outputs retain `eligible_for_simulated_closure` as a legacy Boolean; only the conservative research facade returns the normalized `state` and `documentary_eligible` fields. For research API language, DOCUMENT FOUND means the developer-defined structured evidence checks pass. REVIEW MISSING means the required evidence has not been established. REVIEW CONFLICT means one or more records warrant reconciliation. The conservative research-default module returns these three states; the public demonstration has not been shown to use them. This distinction prevents the paper from claiming that the public demo already implements all proposed status categories.

B.3 Release checklist and source manifest

The research PDF, editable LaTeX source, diagrams, named audits, generated JSON, requirements summary and SHA manifest should accompany one release tag in a future public research repository. The public demo URL can be linked as an adjacent project but must not be used as the only citation for software that is not yet present there. A Zenodo archive DOI can be assigned only after the actual repository release has been uploaded and archived. The current package identifies reproducibility through file content and checksums, not through an invented DOI. External reproduction and a deployed route audit are later tasks.

Technical Appendix C: Planned AI extraction interface

The attached `benchmark_extraction_scaffold.py` consumes two newline-delimited JSON files, one for independently reviewed report labels and one for model predictions. Each line must identify the same case and contain the literal text or extracted fields. Predictions also provide `spans` with verbatim substrings for textual fields. The program counts exact matches, abstentions and missing/invalid supporting spans. It does not train a model, obtain public data or perform inference. It intentionally fails when IDs do not match; that failure is a safeguard against scoring an incomplete prediction collection as if it covered all gold-standard cases.

```
python source/benchmark_extraction_scaffold.py \
  independent_labels.jsonl \
  model_predictions.jsonl
```

A complete subsequent experiment must add separately curated report documents, adjudication provenance, consent or data-access basis, dataset splits by patient/institution, model configuration, prompt/decoding logs, a non-LLM baseline, quality-control rules and prespecified analysis. Without those files, the scaffold cannot produce a meaningful sensitivity, specificity, extraction F1 or clinical risk estimate. A demonstration using labels authored by the same model would recreate the original circularity.

Research integrity and assistance statement

The described dataset and audits contain fictional case information; no patient intervention, real case update, clinical chart review or prospective human-subject experiment is reported. The source-level comparison and finite grid were executed computationally; clinical risk implications are hypotheses requiring external review. Bibliographic and regulatory claims in the text have source links, but this report does not assert professional regulatory advice or a conformity assessment.

AI and editorial assistance: ChatGPT assisted with drafting, editing, code review, debugging, test construction and bibliographic checks. User-supplied critiques attributed to Claude informed the editorial response and testing priorities; those critiques were not treated as independent empirical or clinical adjudication. No language model supplied independent clinical labels. The archived Qwen results were not rerun for this report. Accountability for the report's claims and the decision to release it remains with the author.

Funding and interests: The described software tests were carried out using locally available research tools. No external grant or paid validation engagement is identified in the materials used for this report. ClaimsLens is an author-developed prototype. This technical statement is not an assessment of the author's personal financial interests, and no institutional certification or clinical endorsement is claimed.